# DIY Hi-Fi Pocket Music Player

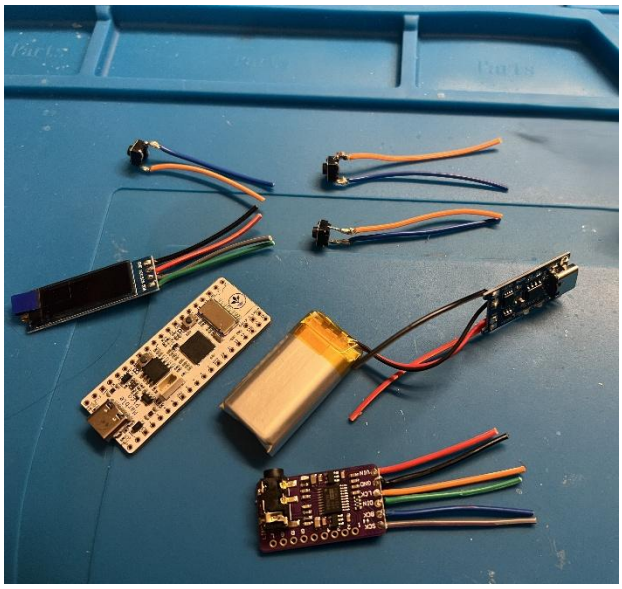
Based on a RP2040 Microcontroller

Ardelean Traian Darius

# 1. Hardware components and specifications

## 1.1 Central Processing unit

- Raspberry Pi Pico (RP2040) - GroundStudio Marble Pico: I opted for this variant because it has an integrated MicroSD slot, which helped me manage the internal space much better.



*Pic. 1 Components*

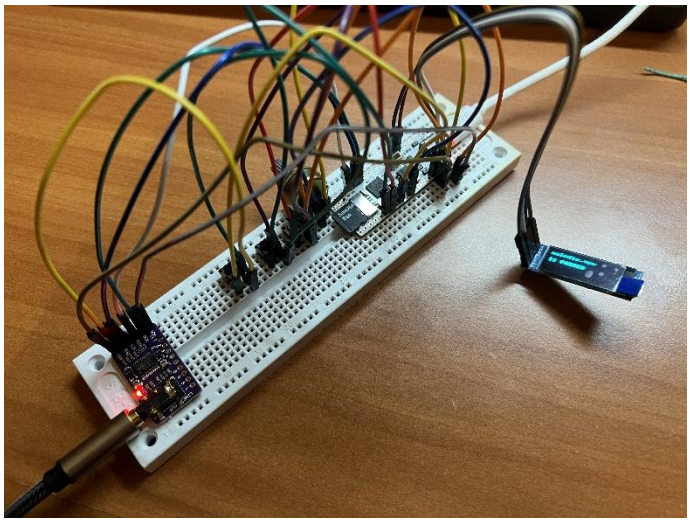
## 1.2 DAC and Display

• PCM5102 DAC: A Digital-to-Analog Converter ensuring Hi-Fi sound, capable of decoding up to 384kHz/32-bit audio.

• OLED 0.96-inch Display SSD1306: A monochrome OLED display with a 128x32 resolution, used for showing device states and track information.

## 1.3 Power and Control

- Li-Po Battery 3.7V (500mAh / 1.85Wh): Model 802035. A small, compact battery that is sufficient for a portable music player.
- TP4056 Charging Module: A modern USB-C charging module featuring overcurrent protection, discharge protection, and 5W charging capabilities.
- User Interface: The interface consists of 3 simple push-down buttons.

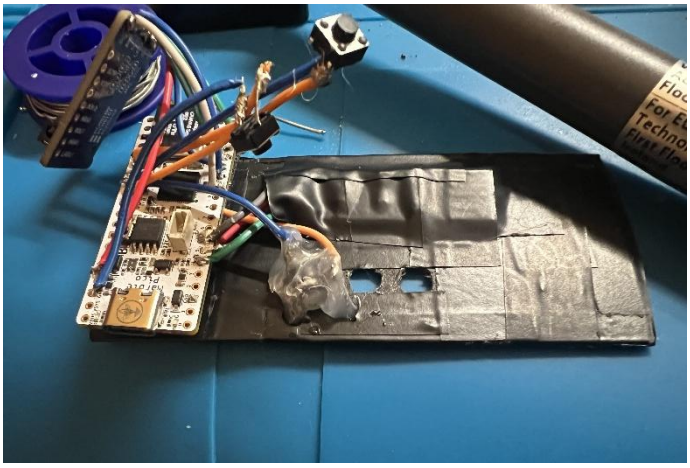


*Pic. 2 Breadboard Testing*

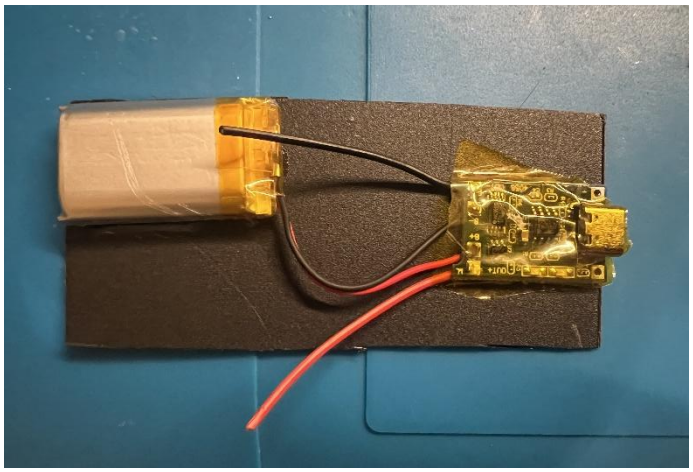
## 2. Structural Design

### 2.1 Custom case

- Used an old DVD case for the front and back plate.
- Consolidated the case by creating "pillars" made of hot glue to prevent the plastic from flexing under pressure.
- Finally, I used electrical tape to cover the edges, protect the internal components, and give it a uniform look.



*Pic. 3 Gluing Components 1*



*Pic. 4 Gluing Components 2*

### 2.2 Gluing the components inside

- After measuring and planning the internal layout, I cut holes in the front plate for the display and buttons.
- I used B-7000 adhesive to secure the battery, velcro tape for initial positioning, and hot glue for the final structural fixing, especially around components with ports that need to withstand pushing and pulling forces.

*Pic. 5 Top side inside*

*Pic. 6 Bottom side inside*

## 3. Ergonomics and ports

- The case is 10 cm wide, 5 cm tall, and 2 cm thick, making it very portable.
- On the front, it features the OLED display and the 3 control buttons.
- On the top side, it houses the ON/OFF switch, the RP Pico USB-C port, and a power LED indicator.
- On the bottom side, it has the 3.5mm audio jack and the MicroSD card slot.
- On the left side, it has the TP4056 USB-C charging port.

*Pic. 7 Front side Final*

*Pic. 8 Bottom side Final*

*Pic. 9  Left side Final*

*Pic. 10 Top side Final*

# 4. Software Logic and Architecture

The software is running on a compatible Raspberry Pi Pico Board, written in C++ using the Arduino IDE environment. It relies on the ESP8266Audio library for audio processing and Adafruit_GFX for the display interface.

## 4.1 Hardware Initialization and Mapping

- External components are connected directly to the Pico Board: the OLED display uses I2C pins 4 (SDA) and 5 (SCL), the DAC connects via I2S to pins 10 (BCK), 11 (LCK), 12 (DIN), GND (SCK), the control buttons utilize digital pins 20, 26, and 21. Storage is managed entirely through the GroundStudio Marble Pico's integrated MicroSD slot.

## 4.2 File Management and Audio Decoding

- Format Filtering: To provide a seamless user experience, the program uses a filtering function during the initialization phase that selectively loads only files with .mp3, .wav, or .flac extensions. Any other files are automatically ignored.
- Buffering and Decoding: Data read from the SD card passes through an 8KB RAM buffer (AudioFileSourceBuffer). This acts as a safety cushion that constantly feeds the audio decoder, eliminating the risk of audio stuttering or lag caused by SD card read speeds (a minimim of Class 10, U1 MicroSD card is required). The system automatically selects the appropriate decoder based on the detected file extension.

## 4.3 GUI and Control Logic

- Geometric Symbols: Instead of standard text characters, the player's status is indicated by geometric symbols drawn pixel-by-pixel: a solid triangle for Play, two vertical rectangles for Pause, and a solid square for Ready/Finished.
- Dynamic Scrolling: For filenames that exceed the 128-pixel width of the OLED screen, a custom horizontal scrolling algorithm was implemented. The text shifts to the left by 2 pixels every 150 milliseconds, allowing the user to read the entire title before the animation resets.
- Time Tracking: Track timing is handled relative to the microcontroller's internal clock (millis()), deliberately avoiding the slow process of parsing complex metadata from audio files. The system calculates the elapsed seconds from the moment a track starts.